

Dataflow Labeling Convention

v3 — adds the **read** verb (scoped) and the datastore **accessModel** axis. **read** is the store → actor counterpart of **write**, permitted only on **direct_access** (passive-storage) datastores; reads from active services still use **pull**. See "Access Model" and section 5.

Purpose

This convention standardizes how dataflows are labeled in Data Flow Diagrams (DFDs).

Goals:

- Improve readability and consistency
- Enable automatic validation
- Make communication semantics explicit
- Support security analysis and threat modeling
- Reduce ambiguity in architecture diagrams

Two-Layer Model

This convention distinguishes two separate layers:

Layer	Content	Effect
Physical Flows	pull , push , write , stream	Define direction and edges in the graph
Logical Annotations	[req_resp], [event_ack]	Mark relationships between flows — do NOT add or remove graph edges

Invariant: Flow direction defines the graph structure. Logical annotations must not introduce or remove graph edges.

Communication Semantics

The verbs in this convention describe communication semantics, not actor perspective, transport protocols, or implementation details.

The model intentionally distinguishes between:

Category	Purpose
pull	Request/response interaction semantics
push	Asynchronous one-way interaction semantics
write	Persistence/state mutation semantics (actor → store)

Category	Purpose
<code>read</code>	Direct passive-storage read semantics — no communication partner (store → actor)
<code>stream</code>	Continuous producer-driven flow semantics

The verbs are intentionally:

- protocol-agnostic
- transport-independent
- actor-neutral

For example:

- `pull` does not mean HTTP GET or database read specifically
- `push` does not mean a transport-layer push protocol
- `write` is not a communication pattern
- `read` is not a communication pattern — it is direct access to passive storage
- `stream` does not imply a specific streaming technology

The convention models interaction semantics rather than implementation mechanics.

A read is modeled according to the datastore's **access model** (see "Access Model" below):

- **Passive storage** (`direct_access`): mutation uses `write`, reading uses `read`. There is no communication partner — both are storage-layer operations.
- **Active service** (`communication`): reading uses `pull` request/response semantics.

Persistence mutations always use `write`, regardless of access model.

Access Model — datastore verb selection

A datastore is either **passive storage** or an **active service**. This is captured explicitly on the datastore element as `accessModel`, and it determines which verbs are legal on flows touching that store.

<code>accessModel</code>	Nature	Read direction	Write direction
<code>direct_access</code>	Passive memory/storage — no responding actor	<code>read</code>	<code>write</code>
<code>communication</code>	Active service that answers requests	<code>pull</code> <code>[req]/[resp]</code>	<code>write</code>

Key distinction: a database server, broker, or cloud store that *answers requests* is semantically an **active service**, not a passive store. Reading it is communication (`pull`). A flash region, shared-memory block, MMIO register, or local file is **passive**; reading it is direct access (`read`).

`accessModel` is **derived from the datastore technology as a default** and may be overridden by the analyst with a rationale (e.g. an in-process SQLite file tagged `database` → `direct_access`; an NFS-mounted `filesystem` → `communication`). **Technology is a default heuristic only — it does not determine the verb.**

`write` is **not** gated by access model: persistence mutation always uses `write`, because it marks a security-relevant effect (durability, retention, integrity, tamper) independent of the access path. `read` **is** gated, because it names an access *pattern* whose semantics depend on whether a responding actor exists.

This makes verb selection deterministic and machine-validatable: the validator resolves the datastore's effective access model and accepts `read/write` only for `direct_access`, and `pull` only for `communication`.

Modelling smell: an `accessModel = "communication"` override on a store-shaped node is the explicit statement that the node is really a fronted service. It surfaces a likely mis-classification (a `Process/ExternalEntity` drawn as a `DataStore`) without forcing a disruptive re-type.

Naming Format

Each dataflow label follows this structure:

```
<verb> <object> [<flow-type>]
```

Where `[flow-type]` is either:

- a **physical flow-type** (`[req]`, `[resp]`, `[cmd]`, `[event]`, `[stream]`), or
- a **logical relationship annotation** (`[req_resp]`, `[event_ack]`)

Examples:

```
pull vehicle state [req]
pull vehicle state [resp]

pull device status [req_resp]

push unlock doors [cmd]
push firmware update applied [event_ack]

write audit log

read input buffer

stream telemetry [stream]
```

Allowed Verbs

Only the following verbs are permitted.

Verb	Meaning	Physical Flow-Types	Logical Annotations
<code>pull</code>	Request/response communication	<code>[req]</code> , <code>[resp]</code>	<code>[req_resp]</code>
<code>push</code>	One-way asynchronous action, command, or event	<code>[cmd]</code> , <code>[event]</code>	<code>[event_ack]</code>
<code>write</code>	Persist data into a datastore	none	none
<code>read</code>	Read directly from a passive datastore (<code>direct_access</code> only)	none	none
<code>stream</code>	Continuous or repeated data flow	<code>[stream]</code> (optional)	none

Deprecated / Forbidden Verbs

The following verbs must not be used:

```
send
recv
```

Also avoid synonyms such as:

```
fetch
query
emit
notify
publish
receive
```

Note: `read` is **no longer forbidden** — it is an allowed verb, but **scoped**: it is valid only on `direct_access` datastores (passive storage), in the store → actor direction. Reads from active services/repositories still use `pull`. See "Access Model" and section 5 (`read`).

The goal is to keep the vocabulary intentionally small and semantically consistent.

Rules by Verb

1. pull

Meaning

pull represents a request/response interaction.

pull describes the semantics of a logically coupled request/response interaction, independent of protocol or transport implementation.

The term is actor-neutral: the graph direction defines the actual flow direction, while **pull** identifies the interaction pattern itself.

Typical examples include:

- API calls
- Database queries
- RPC communication
- Service lookups
- Request/response protocols

Physical Flow-Types

Flow-Type	Meaning
[req]	The outgoing request half of a request/response pair
[resp]	The incoming response half of a request/response pair

Logical Annotation

Annotation	Meaning
[req_resp]	Compact notation representing a logically coupled request/response pair on a single flow

[req_resp] does not create a second graph edge. It is a modeling shorthand indicating that the flow logically represents a paired interaction.

Required Usage Patterns

A **pull** flow MUST use one of the following patterns:

Option 1 — Explicit pairing (recommended for security analysis)

```
pull <object> [req]      (A → B)
pull <object> [resp]     (B → A)
```

Both directions must be modeled as separate graph edges. The response must use the same object and reverse the direction of the request.

Option 2 — Compact representation

```
pull <object> [req_resp]
```

Represents the logically coupled pair on a single edge. Choose this when the response edge is implicit or does not require separate analysis.

Examples

Valid

```
pull vehicle state [req]
pull vehicle state [resp]

pull safety params [req]
pull safety params [resp]

pull device status [req_resp]
```

Invalid

pull vehicle state	← missing flow-type or annotation
pull vehicle state [cmd]	← [cmd] is not valid for pull
pull vehicle state [event]	← [event] is not valid for pull

2. push

Meaning

push represents a one-way asynchronous flow.

Typical examples:

- Commands
- Events
- Fire-and-forget messages
- Notifications

push is always unidirectional. Acknowledgement relationships are modeled via the **[event_ack]** logical annotation, not via a second **push** flow.

Physical Flow-Types

Flow-Type	Meaning
[cmd]	An action or instruction intended to trigger behavior
[event]	A notification that something already happened

Logical Annotation

Annotation	Meaning
[event_ack]	An event that is expected to be acknowledged via a correlated flow

[event_ack] does not represent a different type of event. It is a modeling annotation indicating that an acknowledgement relationship exists. It does NOT imply bidirectional communication and does NOT add a graph edge.

Examples

Valid Commands

```
push unlock doors [cmd]
push refresh cache [cmd]
push rotate keys [cmd]
```

Valid Events

```
push obstacle detected [event]
push vehicle moved [event]
push user authenticated [event]
```

Valid Events with acknowledgement annotation

```
push firmware update applied [event_ack]
push configuration changed [event_ack]
push safety mode activated [event_ack]
```

Invalid

```
push unlock doors          ← missing flow-type
push unlock doors [req]    ← [req] is not valid for push
push unlock doors [resp]   ← [resp] is not valid for push
```

3. write

Meaning

write represents persistence into a datastore or repository.

write exists because persistence introduces architectural and security semantics distinct from communication flows.

Examples include:

- durability
- auditability
- retention
- integrity boundaries
- compliance-relevant persistence

Reads are not modeled using **write**. How a read is modeled depends on the datastore's access model: direct reads from passive storage use **read**; reads from an active service/repository use **pull**. See "Access Model" and section 5 (**read**).

For example, a read from an active repository service:

```
pull user profile [req]
pull user profile [resp]
```

Typical examples:

- Logging
- Audit persistence
- Database writes
- State persistence

write is not part of the communication model. It represents a persistence operation, not a message exchange. **write** does not participate in **req_resp** or **event_ack** semantics.

Rules

write MUST NOT use any flow-type or logical annotation.

Examples

Valid


```
write audit log
write vehicle state
write telemetry record
```

Invalid

```
write audit log [cmd]      ← no flow-type allowed
write telemetry [req]      ← no flow-type allowed
```

4. stream

Meaning

stream represents a continuous or repeated producer-driven data flow.

Typical examples:

- Video feeds
- Sensor streams
- Telemetry streams
- Continuous monitoring data

stream flows are independent. They do not participate in **req_resp** or **event_ack** semantics. If a control channel exists alongside a stream (e.g. start/stop commands), it MUST be modeled as a separate **push [cmd]** flow. Responses to control commands (e.g. start failed, stream ended) MUST be modeled as **push [event]** or **pull [resp]** — never as part of the stream flow itself. Stream control flows belong to the **push/pull** model, not to the stream flow.

Flow-Type

[stream] is optional and may be added for clarity.

stream MUST NOT use **[req]**, **[resp]**, **[cmd]**, **[event]**, **[req_resp]**, or **[event_ack]**.

Examples

Valid

```
stream camera frames
stream telemetry
stream radar objects [stream]
```

Invalid

```
stream telemetry [req]      ← not valid for stream
stream telemetry [cmd]      ← not valid for stream
stream telemetry [event_ack] ← stream does not participate in ack
semantics
```

5. read

Meaning

read represents a **direct read from a passive datastore** (load/store, syscall, memory-mapped access).

read exists because some storage access has **no communication semantics** — there is no requesting/responding actor, only a memory or storage region that is read directly. Modelling such access as **pull** would describe a request/response channel that does not exist, and would generate the wrong threat set (request spoofing, response tampering) while missing the real one (TOCTOU, race conditions, residual data exposure, region access).

read is the store → actor counterpart of **write**. Both are storage-layer operations; neither participates in the communication model.

Typical examples:

- Shared memory between cores (ARM ↔ DSP, dual-port RAM, mailbox)
- Memory-mapped registers / MMIO
- DMA / ring / scratchpad buffers
- Direct reads from Flash / EEPROM / NVRAM
- Reads from a local file (in-process), not via a service

Reads from an **active** storage service (database server, cloud store, broker) are NOT modelled with **read** — they are request/response interactions and use **pull**.

Access-model gate

read is permitted **only** on a datastore whose access model is **direct_access** (passive memory/storage). On a datastore whose access model is **communication** (active service), the read direction MUST use **pull**. See "Access Model".

write is NOT gated: persistence mutation always uses **write** regardless of access model, because it marks a security-relevant effect (durability, retention, integrity, tamper) independent of the access path.

Rules

read MUST NOT use any flow-type or logical annotation. **read** is directional: **store** → **actor** (the inverse of **write**, which is actor → store).

Examples

Valid

```
read input buffer
read fft result
read calibration block
```

Invalid

```
read fft result [req]      ← no flow-type allowed
read fft result [req_resp] ← read does not participate in
pairing
read user profile          ← if the store is an active service →
use pull
```

Logical Relationship Annotations

Logical annotations are syntactic macros over physical flows. They are expanded deterministically before analysis. They may appear on a diagram edge but do not change the underlying graph semantics.

Tooling requirement: Expansion MUST occur before any graph-based analysis or security evaluation (e.g. threat generation, attack path tracing, permission boundary checks).

req_resp

Represents a logically coupled request/response pair as a single diagram notation.

In the diagram: one edge. In analysis: expands to two directed physical edges:

```
pull <object> [req]    (A → B)
pull <object> [resp]   (B → A)
```

Both edges are analyzed independently for threats.

Valid only on **pull** flows.

```
pull <object> [req_resp]
```

event_ack

Represents a logical dependency indicating that an acknowledgement flow exists or is expected for this event.

It does not create a physical flow and does not change the graph structure. The acknowledgement is a semantic relationship only. If the acknowledgement requires explicit security analysis, model it as a separate **push** [event] or **pull** [resp] flow instead.

Valid only on **push** flows.

```
push <object> [event_ack]
```

Expansion Rules

Logical annotations are processed before any graph-based analysis or security evaluation. Processing differs by annotation type:

Annotation	Processing	Result
[req_resp]	Expands into two directed physical edges	Two graph edges for analysis
[event_ack]	Resolved as semantic dependency only	No new graph edge

req_resp expansion

```
pull <object> [req_resp]
```

expands to:

```
pull <object> [req]      (A → B)
pull <object> [resp]     (B → A)
```

Both directions are analyzed independently for threats. Each expanded edge MUST be analyzed independently for security and threat modeling purposes.

event_ack — no expansion

event_ack does NOT expand into physical flows.

It represents a semantic dependency only: the event is expected to be acknowledged, but the acknowledgement is not a physical flow in this model.

If the acknowledgement itself requires security analysis, it **MUST** be modeled as an explicit separate flow using `push [event]` or `pull [resp]` — at which point `[event_ack]` is no longer needed on the original flow.

Object Naming Guidelines

The object should describe the data or domain concept being transferred.

Prefer:

```
vehicle state
safety params
audit log
camera frames
```

Avoid:

```
get vehicle state    ← verb embedded in object
mqtt payload         ← transport detail
json message         ← implementation detail
send data            ← meaningless
```

The object should:

- be concise
- be domain-oriented
- avoid transport/protocol details
- avoid implementation-specific wording

Validator rule: The object **MUST NOT** contain transport or encoding terms. Forbidden terms include: `json`, `mqtt`, `http`, `grpc`, `rest`, `payload`, `message`, `packet`, `frame`, `buffer`.

Good Examples

```
pull user profile [req]
pull user profile [resp]

pull device status [req_resp]

push rotate keys [cmd]
push firmware update applied [event_ack]
```

```
write security audit entry  
  
read shared sensor buffer  
  
stream radar objects [stream]
```

Bad Examples

send user profile	← forbidden verb
recv user profile	← forbidden verb
 pull	← missing object and flow-type
push [cmd]	← missing object
 write config [cmd]	← write must not have flow-type
stream telemetry [req]	← req not valid for stream
push event [req_resp]	← req_resp not valid for push

Design Principles

This convention intentionally uses a very small vocabulary.

Benefits:

- Easier to read
- Easier to validate automatically
- Easier to model security semantics
- Consistent architecture diagrams
- Reduced ambiguity

The label communicates:

- communication style
- direction semantics
- persistence intent
- synchronization behavior

without requiring additional explanation.

The convention intentionally separates:

- communication semantics (**pull**, **push**, **stream**)
- persistence semantics (**write**)

This avoids mixing interaction patterns with storage operations.

Hard Invariants

1. **Flow direction defines graph structure.** Logical annotations (`[req_resp]`, `[event_ack]`) must not introduce or remove graph edges.
2. **A flow may be labeled with either a physical flow-type or a logical shorthand annotation — not both.** Logical annotations are syntactic sugar and are expanded during analysis. They may appear on the diagram edge but do not change the underlying graph semantics.
3. **read, write, and stream do not participate in the pairing system.** They have no `req_resp` or `event_ack` semantics. `write` (actor → store) represents persistence mutation; `read` (store → actor) represents direct passive-storage access; neither is part of the communication graph. `read` is permitted only on `direct_access` datastores; `write` is permitted on any datastore.

Formal Grammar

```
Flow      ::= Verb Object [ Tag ]
Verb      ::= "pull" | "push" | "write" | "read" | "stream"
Object    ::= word { " " word }
Tag       ::= PhysicalTag | LogicalTag
PhysicalTag ::= "[req]" | "[resp]" | "[cmd]" | "[event]" | "[stream]"
LogicalTag  ::= "[req_resp]" | "[event_ack]"
```

Constraints not expressible in EBNF are enforced by validation rules:

- `write` MUST NOT carry any tag
- `read` MUST NOT carry any tag
- `read` is valid only on a flow whose datastore endpoint has `accessModel = direct_access`, in the store → actor direction
- `write` is valid in the actor → store direction on any datastore
- `stream` MUST NOT carry `PhysicalTag` other than `[stream]`, and MUST NOT carry `LogicalTag`
- `pull` MUST carry exactly one of: `[req]`, `[resp]`, `[req_resp]`
- `push` MUST carry exactly one of: `[cmd]`, `[event]`, `[event_ack]`

Processing Pipeline

Tools implementing this convention MUST follow this execution order:

1. Parse — tokenize and validate label syntax against grammar
2. Validate — enforce verb/tag constraint rules
3. Process — expand `[req_resp]` into physical edges only; attach `[event_ack]` as metadata only (no graph mutation)
4. Build — construct directed graph from physical flows only

5. Analyze – run threat generation, attack path tracing, boundary checks

After step 3, all logical annotations are resolved:

- `[req_resp]` → eliminated via expansion into two directed physical edges
- `[event_ack]` → removed from graph layer; carried as metadata only (no graph representation)

The graph passed to step 4 and beyond contains physical flows only.

Summary

Physical Flows

Verb	Flow-Types	Notes
<code>pull</code>	<code>[req]</code> , <code>[resp]</code>	Explicit request/response pair
<code>push</code>	<code>[cmd]</code> , <code>[event]</code>	One-way asynchronous
<code>write</code>	none	Persistence mutation (actor → store); any datastore
<code>read</code>	none	Direct passive-storage read (store → actor); <code>direct_access</code> only
<code>stream</code>	<code>[stream]</code> (optional)	Continuous producer-driven

Logical Annotations

Annotation	Valid On	Meaning
<code>[req_resp]</code>	<code>pull</code> only	Compact request/response pair notation
<code>[event_ack]</code>	<code>push</code> only	Event with expected acknowledgement

pull

```
pull <object> [req]
pull <object> [resp]
pull <object> [req_resp]
```

push

```
push <object> [cmd]
push <object> [event]
```



```
push <object> [event_ack]
```

write

```
write <object>
```

read

```
read <object>
```

Valid only on `direct_access` datastores, store → actor.

stream

```
stream <object>  
stream <object> [stream]
```